NeuraLSP: An Efficient and Rigorous Neural Left Singular Subspace Preconditioner for Conjugate Gradient Methods

Anonymous Authors[1]

Abstract

Numerical techniques for solving partial differential equations (PDEs) are integral for many fields across science and engineering. Such techniques usually involve solving large, sparse linear systems, where preconditioning methods are critical. In recent years, neural methods, particularly graph neural networks (GNNs), have demonstrated their potential through accelerated convergence. Nonetheless, to extract connective structures, existing techniques aggregate discretized system matrices into graphs, and suffer from rank inflation and a suboptimal convergence rate. In this paper, we articulate NeuraLSP, a novel neural preconditioner combined with a novel loss metric that leverages the left singular subspace of the system matrix's near-nullspace vectors. By compressing spectral information into a fixed low-rank operator, our method exhibits both theoretical guarantees and empirical robustness to rank inflation, affording up to a 53% speedup. Besides the theoretical guarantees for our newly-formulated loss function, our comprehensive experimental results across diverse families of PDEs also substantiate the aforementioned theoretical advances. Our code is publically available at `https://github.com/alexbenanti/NeuraLSP`.

1. Introduction

Background and Major Challenges: Partial differential equations (PDEs) are quintessential to various computational problems in science, engineering, and related applications, including simulation, modeling, and scientific computing. PDEs are typically solved numerically as large, sparse linear systems in the form of $\mathbf{Au} = \mathbf{f}$. Modern linear algebra routines, such as the algebraic multigrid (AMG) method, significantly accelerate these computations by combining smoothing with coarse-grid error corrections (Stüben, 2001; Falgout, 2006; Ruge & Stüben, 1987; Xu & Zikatanov, 2017; Brandt, 1977; Briggs et al., 2000; Trottenberg et al., 2001). However, traditional AMG solvers rely on large, heuristic prolongation matrices, and thus have difficulty reducing the smooth error vectors, which are the near-nullspace components of the system matrix $\mathbf{A}$. This results in excessive computational overhead in obtaining numerical solutions. Preconditioners can also help accelerate the process, but their selection is heuristic and their construction is expensive. In contrast, recent deep neural methods, particularly graph neural networks (GNNs), have the potential to leverage big data and learn optimal preconditioners and prolongation operators. However, many currently available neural methods treat deep networks as black boxes. These methods generally face unresolved challenges, including a lack of a sound mathematical foundation, no guarantee of correctness or convergence, and low accuracy and efficiency.

Motivation and Method Overview: We observe that the prolongation matrix is typically constructed from the strength of connection of the matrix graph. This raises a key question that must be properly addressed: Can we construct a smaller, coarser subspace via machine learning while maintaining comparable convergence rates? In this paper, we present NeuraLSP, a robust preconditioned multigrid solver, which is outlined in Fig. 2. NeuraLSP is built upon our neural low-rank preconditioners (obtained via an application of Algorithm 1), which are optimized with our novel nested loss metric that leverages the left singular subspace. The hallmark of NeuraLSP is its provable theoretical completeness and empirical robustness to ***rank inflation*** – a bottleneck commonly observed in conventional AMG methods that highlights a need for an unnecessarily large coarse space. Thorough mathematical analysis and extensive experimental verification demonstrate the effectiveness of our method. To the best of our knowledge, this paper showcases ***the first research attempt*** to leverage a learned left singular subspace to conquer rank inflation.

Major Contributions: The salient contributions of this paper comprise: (1) ***Rigorous neural architecture***. We present

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

NeuraLSP, a novel neural preconditioned multigrid framework grounded in mathematical rigor. It uses learned left singular vectors to construct fixed low-rank preconditioners. NeuraLSP is provably robust to rank inflation, and affords high efficiency, high accuracy, and strong generalization power to multiple PDE variants; (2) ***Insightful loss metric***. We propose our novel NLSS loss that efficiently recovers the leading left singular subspace of the system matrix's near-nullspace vectors. Our subspace loss significantly reduces the size of the coarse space, potentially leading to computational speedups upon convergence; and (3) ***Extensive experiments***. We demonstrate that our method reduces solve time when compared to both classical and neural PDE solvers across multiple benchmarks. Extensive experiments, ablations, and evaluations have verified all of our claimed advantages and confirmed that our proposed method outperforms the state-of-the-art (SOTA).

2. Related Work

Legacy SVD Solvers based on the Near-Nullspace: The multigrid method's key component is the *prolongation matrix*, which maps coarse corrections back to the fine level. It is also needed in the Galerkin projection to construct the *restriction matrix*, $\mathbf{R}$, which maps the fine residual/error to the coarse level. The algebraic multigrid (AMG) paradigm (Stüben, 2001; Falgout, 2006; Ruge & Stüben, 1987; Xu & Zikatanov, 2017) effectively handles unstructured domains, such as irregular grids and meshes. However, the smooth error in AMG cannot be further reduced by any relaxation methods. These *smooth error* vectors are defined as the *near-nullspace* components of the system matrix $\mathbf{A}$. The error in AMG is smooth along the direction of strong connections in $\mathbf{A}$. The prolongation matrix in AMG is computed from the strength of connection of the matrix graph. For simple two-grid correction schemes, the prolongation matrix can be relatively large, thereby slowing computations through large matrix multiplications.

While classical AMG focuses on selecting individual coarse points, the smoothed-aggregation algebraic multigrid (SA-AMG) focuses on clustering nodes together and then smoothing the transitions between them (Vaněk et al., 1996). The critical distinguishing factor is the use of near-nullspace vectors. The collection of smooth vectors, $\mathbf{S}$, is passed into the solver, and the prolongation operator is constructed for the specific purpose of preserving these modes. However, this idea of utilizing the near-nullspace has been extended beyond AMG. Most notably, (Chow, 2006) computes the local SVD of the smooth error aggregates and uses the left singular vectors for prolongation and the singular values/right singular vectors to help accelerate smoothing for the next lower level in the multigrid process.

Neural SVD Solvers: Given the rise in prominence of spectral information across the research landscape in recent years, it is necessary to explore this direction to help solve our problem. Backpropagation-friendly eigendecomposition is introduced in (Wang et al., 2019). Related work amortizes the cost of eigendecomposition via improved objectives and QR-decomposition (Li et al., 2024). Stabilization of GANs using spectral normalization is also discussed in (Miyato et al., 2018). The use of SVD in 3D rotations in neural networks is heavily explored in (Levinson et al., 2020). Learning the spectral information of operators is discussed in works such as (Pfau et al., 2019; Shaham et al., 2018; Deng et al., 2022). Finally, specific algorithms and objectives are developed to identify the leading principal components in works such as (Bao et al., 2020; Oftadeh et al., 2020; Gemp et al., 2021). So, it is prudent to seek new integrated solutions (built on both machine learning and spectral information) that are significant for solving PDEs.

The work of (Chow, 2006) presents a promising approach to using SVD to help construct the prolongation operator. A significant drawback of this approach is the expensive computation of the SVD $\mathcal{O}(\min(m,n)mn)$ (Golub & Van Loan, 2013). Thus, we explore low-rank approximations to extract sufficient information while minimizing computational cost. This has been done in (Eckart & Young, 1936) by formulating the best rank-k approximation. The work is further developed using randomized algorithms in (Halko et al., 2011).

With the rise of scientific machine learning, it is imperative to seek neural methods towards constructing the prolongation matrix. In fact, this idea has been explored by (Greenfeld et al., 2019) for structured grids and further extended by (Luz et al., 2020) for unstructured grids using GNNs. This idea has been explored in the continuous setting by (Ryu et al., 2024). If we can capture sufficient near-nullspace information from the SVD, we can significantly reduce the size of the coarse space, potentially leading to computational speedups. The main contributions of our work, which are highlighted in the previous section, emphasize that this improvement can be realized.

Our work has a variety of applications across science and engineering. For example, the utilization of smoothed vectors for obtaining a multigrid preconditioner of Wilson fermions in lattice QCD is shown in (Whyte et al., 2025). Our method can also be extended to multigrid techniques used for structural mechanics/elasticity problems (Vanek et al., 1999; Griebel et al., 2003); incompressible computational fluid dynamics/Navier-Stokes problems (Knoll & Mousseau, 2000; Pernice & Tocci, 2001; Elman et al., 2003); and electromagnetics/Maxwell problems (Reitzinger & Schöberl, 2002; Bochev et al., 2003; Hiptmair & Xu, 2007).

3. Mathematical Preliminaries

3.1. Linear Systems and PDEs

Generally, we can define a PDE with Dirichlet boundary conditions as follows:

$$\begin{cases} \mathcal{D}[u(\mathbf{x})] = f(\mathbf{x}), & \mathbf{x} \in \mathbf{\Omega}, \\ u(\mathbf{x}) = 0, & \mathbf{x} \in \partial\mathbf{\Omega}, \end{cases} \tag{1}$$

where $\mathcal{D}$ is a differential operator, $\mathbf{\Omega} \subset \mathbb{R}^n$ is a bounded domain with $\partial\Omega$ being the boundary. However, many PDEs lack analytical solutions and must therefore be solved numerically. This is typically done by reformulating the problem as a linear system:

$$\mathbf{A}\mathbf{u} = \mathbf{f}, \tag{2}$$

where $\mathbf{A} \in \mathbb{R}^{n\times n}$, $\mathbf{u}, \mathbf{f} \in \mathbb{R}^n$. There are different ways to construct $\mathbf{A}$ depending on the method used. For regularly shaped grids, we can use finite-difference methods (Thomas, 1995); for conservation methods, finite-volume methods (LeVeque, 1992); and, for irregularly shaped grids, finite-element methods (Brenner & Scott, 2008).

3.2. AMG and Smoothed Aggregation

AMG ignores the physical geometry and looks only at the matrix $\mathbf{A}$. To do this, we treat the variables as nodes in a graph and the non-zero matrix entries as edges. We coarsen by grouping variables that are "strongly-coupled" (largely negative off-diagonal entries) into aggregates. This is an ideal choice when node connectivity is irregular.

For symmetric positive definite (SPD) problems, a popular way of constructing coarse grids in AMG is the Smoothed Aggregation algorithm. The idea is to group fine-grid nodes into aggregates, as before, so that the domain is physically decomposed into small patches. Then, we can construct a piecewise-interpolation matrix $\mathbf{T}$ that preserves the "near-nullspace" of the operator (i.e., vectors that the operator does not change much). The critical step is then taking $\mathbf{T}$ and applying one step of a smoothing method (usually weighted Jacobi) to "blur" any sharp edges that may appear and use it to construct $\mathbf{P}$: $\mathbf{P} = (\mathbf{I} - \omega\mathbf{D}^{-1}\mathbf{A})\mathbf{T}$, so that we efficiently lower the energy of the basis functions that would have previously been high due to sharp jumps in $\mathbf{T}$. $\omega \in [0, 1]$ is known as the *relaxation parameter* that controls how much of the old solution is combined with the update. This means that the columns of $\mathbf{P}$ now overlap so that a coarse node does not influence only its own aggregate; it also spreads its influence to its neighbors. This process, when followed by multigrid, is referred to as SA-AMG.

3.3. Near-Nullspace Vectors

Similarly to bootstrap AMG (Brandt et al., 2011), we can also generate near-nullspace vectors simply by starting from an initial random collection of vectors $\mathbf{S}^{(\mathbf{0})} \in \mathbb{R}^{n\times K}$ and then applying a smoothing algorithm to it, i.e., with Jacobi smoothing: $\mathbf{S}^{(\mathbf{s_1})} = (\mathbf{I} - \omega\mathbf{D}^{-1}\mathbf{A})^{s_1}\mathbf{S}^{(\mathbf{0})}$. When we construct the near-nullspace vectors in this manner, the columns of $\mathbf{S}^{(\mathbf{s_1})}$ concentrate in a low-dimensional subspace associated with smooth-error and near-nullspace components. Like in SA-AMG, the purpose of $\mathbf{P}$ is to represent this subspace well so that, when coarse-correction is applied, the smooth error will be eliminated efficiently.

3.4. SVD for Constructing P

When we generate these initial vectors randomly, there isn't much "structure" immediately apparent in our complete matrix $\mathbf{S} \equiv \mathbf{S}^{(\mathbf{s_1})}$ because we did not previously divide our operator up into nodes/aggregates. However, this does *not* imply that we still cannot extract useful information from $\mathbf{S}$ to construct $\mathbf{P}$.

Consider the SVD of $\mathbf{S}$, $\mathbf{S} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$. The work of (Chow, 2006) highlights that we can seek $\mathbf{P}$ as a rank-k optimal basis for $\mathbf{S}$, i.e., formally, we want $\min_{\mathbf{P},\mathbf{W}} ||\mathbf{S} - \mathbf{P}\mathbf{W}||_2$, where $|| \cdot ||_2$ is the matrix spectral norm. This minimum is achieved when $\mathbf{P}\mathbf{W} = \mathbf{U_k}\mathbf{\Sigma_k}\mathbf{V_k}^T$ and, by matching variables, we find that $\mathbf{P} = \mathbf{U_k}$. This makes sense as $\mathbf{U_k}$ will span the dominant subspace of the smoothed error samples. It is important to note that the factorization is not unique. In fact, we have $\mathbf{P} = \mathbf{U_k}\mathbf{Q}$ and $\mathbf{W} = \mathbf{Q}^T\mathbf{\Sigma_k}\mathbf{V_k}^T$ for any orthogonal $\mathbf{Q} \in \mathbb{R}^{k\times k}$. In reality, only the subspace span($\mathbf{P}$) is defined. It is important to note that in (Chow, 2006), the work was conducted when $\mathbf{S}$ was partitioned into aggregates to keep SVD computations small. However, what if there were a way to bypass this aggregation process and avoid heavy SVD computations? In the next section, we replace the per-instance SVD of the smoothed-vector matrix $\mathbf{S}$ with a learned map that produces an orthonormal coarse basis $\mathbf{P}$ while preserving the key subspace recovery guarantee at global optima.

4. Novel Method

Given a PDE as described in Eq. (1), we discretize it in the form Eq. (2) using finite element methods and use a two-grid scheme to construct a preconditioner for the system so that iterative methods may be used to solve it. NeuraLSP consists of two critical components: (a) our novel nested loss metric with theoretical guarantees (Sec. 4.1), and (b) our novel architecture, which integrates mathematical rigor and machine learning into one concrete pipeline (Sec. 4.2).

4.1. NLSS Loss with Theoretical Guarantees

Theoretical Insights: We aim to improve the work of (Chow, 2006) by amortizing the cost of computing the SVD of a given matrix using deep learning. We observe that to

construct the actual prolongation matrix, $\mathbf{P}$, the left singular vectors of the collection of smoothed vectors, $\mathbf{S}$, are all we need. Thus, our objective is to construct a loss function that learns only the left singular subspace up to a specified rank k.

One may observe that a natural choice is to maximize the *Subspace Reconstruction Loss*, which is given as follows:

$$\mathcal{L}(\mathbf{U}) = ||\mathbf{U}\mathbf{U}^T\mathbf{S}||_F^2 = \text{Tr}(\mathbf{U}^T\mathbf{S}\mathbf{S}^T\mathbf{U}). \quad (3)$$

However, because Eq. (3) is invariant under any orthogonal change of basis in the columns of $\mathbf{U}$, it cannot distinguish orderings of the same subspace. So, it is a natural next step to address this limitation. We were inspired by the work done in (Ryu et al., 2024), where the authors used a LoRA-style loss (Hu et al., 2021) to learn the singular functions of continuous operators. In our case, our aim is to adapt this idea to discrete operators. Our proposed *Neural Left Singular Subspace* (NLSS) loss function is given as follows:

$$\mathcal{L}^{(k)}(\mathbf{S}, \tilde{\mathbf{P}}(\boldsymbol{\theta}); \boldsymbol{\theta}) = \frac{1}{k}\sum_{\ell=1}^{k}\left(1 - \frac{||\tilde{\mathbf{P}}_\ell^T\mathbf{S}||_F^2}{||\mathbf{S}||_F^2}\right), \quad (4)$$

where $\tilde{\mathbf{P}}_\ell$ represents the first ℓ columns of the matrix $\tilde{\mathbf{P}} \in \mathbb{R}^{n\times k}$. This loss is designed to accurately learn the first k singular vectors of the matrix $\mathbf{S}$. That is, if $\mathbf{U_k}\mathbf{\Sigma_k}\mathbf{V_k}^T$ is the truncated SVD of $\mathbf{S}$, we should have $\tilde{\mathbf{P}} \approx \mathbf{U_k}$. We demonstrate this in the following theorem (see the proof in the appendix).

Theorem 4.1. *Let $\mathbf{S} \in \mathbb{R}^{n\times m}$ and let $\tilde{\mathbf{P}} = [\tilde{\mathbf{p_1}}, \cdots, \tilde{\mathbf{p_k}}] \in St(n,k)$ be a matrix with orthonormal columns ($\tilde{\mathbf{P}}^T\tilde{\mathbf{P}} = \mathbf{I_k}$) where $St(n,k)$ is the Stiefel manifold of orthonormal $n \times k$ matrices. Suppose that the SVD of $\mathbf{S} \in \mathbb{R}^{n\times m}$ is given as $\mathbf{S} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$. Then, any global minimizer $\mathbf{P}^* = \arg\min_{\tilde{\mathbf{P}}} \mathcal{L}^{(k)}(\mathbf{S}, \tilde{\mathbf{P}})$ has columns that span the leading k-dimensional left singular subspace of $\mathbf{S}$ where $\mathcal{L}^{(k)}$ is defined from Eq. (4).*

Theorem 4.1 presents an additional strong result that we highlight in Corollary 4.2. The idea is that we can recover the actual leading k left singular vectors up to signs using our loss function. We present the proof in the appendix.

Corollary 4.2. *Let $\mathbf{S} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T \in \mathbb{R}^{n\times m}$ where $\mathbf{\Sigma} = diag(\sigma_1, \cdots, \sigma_m)$. If $\sigma_1 > \sigma_2 > \cdots > \sigma_k > \sigma_{k+1}$, then any orthonormal minimizer $\mathbf{P}^* \in \mathbb{R}^{n\times k}$ of Eq. (4) is equal to $\mathbf{U}_k$ up to column-wise signs.*

Thus, the NLSS loss facilitates a neural model where the subspace dimension k is a tunable parameter, allowing direct control over the spectral information retained. This contrasts with standard AMG implementations, where the coarse space size is determined by fixed heuristics (e.g., strength of connection) rather than user-defined constraints. See Fig. 1.

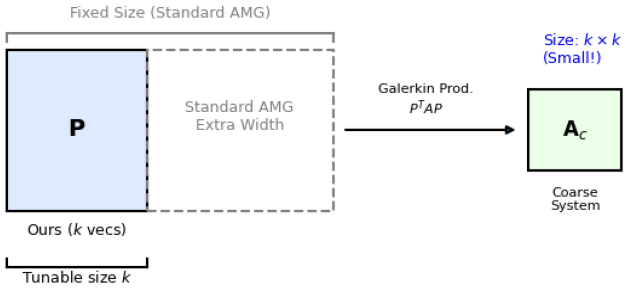

Figure 1. A visual demonstration of how our prolongation matrix can be adjusted based on how much spectral information we decide to capture.

Two-Grid Convergence Guarantees: Let $\mathbf{A}$ be an SPD matrix and $\mathbf{e}$ denote the error after pre-smoothing. By construction, the columns of $\mathbf{S}$ sample the "smooth" error modes that are not effectively reduced by the smoother. For any error component that lies strictly in the coarse space, $\mathbf{e} \in \text{Range}(\mathbf{P})$ (i.e., $\mathbf{Py} = \mathbf{e}$), the update becomes: $\mathbf{e}_{\text{new}} = \mathbf{E}_{\text{TG}}\mathbf{e}_{\text{old}} = \mathbf{Py} - \mathbf{P}(\mathbf{P^TAP})^{-1}(\mathbf{P^TAP})\mathbf{y} = \mathbf{Py} - \mathbf{Py} = \mathbf{0}$. (For simplicity, we omitted the pre- and post-smoothing steps.) Although this exact elimination holds for any Galerkin projection, the effectiveness of the preconditioner will depend on how much of the *total* smooth error is contained within range($\mathbf{P}$).

Thus, the result of Theorem 4.1 enables us to construct an effective two-grid preconditioning scheme for SPD problems by minimizing total energy. The span of the global minima of the NLSS loss guarantees the maximum possible reduction of smooth error energy achievable by any rank-k preconditioner. We provide a proof in the appendix, which is aided by the work of (Eckart & Young, 1936; Fan, 1949).

This process can be used as a preconditioner for Krylov-based methods. If the smoothed test vectors $\mathbf{S}$ are enriched in algebraically smooth error components (as standard smoothing heuristics suggest), then capturing their dominant subspace produces a coarse space that targets the slow-to-converge modes in Krylov iterations (Frank & Vuik, 2001).

4.2. Neural Network Design

Although we have shown that the loss can accurately learn the left singular subspace of $\mathbf{S}$ up to a specified rank, we must still be careful when designing the learning algorithm. That is, we assumed that $\tilde{\mathbf{P}}$ contains orthonormal columns. To ensure the final output has orthonormal columns in accordance with the assumptions made in Theorem 4.1, we perform a QR-decomposition on the learned matrix $\tilde{\mathbf{P}}$ before outputting it from the neural network.

Two-Level Subspace Correction: We outline the algorithm for the two-level design in Algorithm 1. To keep the steps

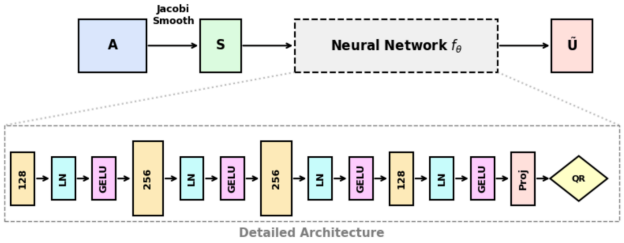

Figure 2. Our NeuraLSP pipeline and architecture overview. The system matrix $\mathbf{A}$ is smoothed into a collection of vectors $\mathbf{S}$, which is passed through a neural network consisting of a 4-layer MLP (widths 128-256-256-128) with LayerNorms and GELU activations in-between. Then, the output is projected onto a matrix, and QR-decomposed to ensure orthonormality. The result is the left singular subspace $\tilde{\mathbf{U}}$.

simple, for the traditional two-level steps, we follow the steps provided in (Luz et al., 2020). However, for a more detailed overview of the algorithm, we suggest that the reader examines (Briggs et al., 2000). We have also included a high-level illustration of the NeuraLSP pipeline in Fig. 2 for constructing the prolongation matrix. It is important to note that Algorithm 1 describes only one application of our two-level preconditioner; in experiments, we apply it to preconditioned conjugate gradient (PCG).

Algorithm 1 Two-Level Subspace Correction

Input: Coefficient matrix: $\mathbf{A} \in \mathbb{R}^{n\times n}$, matrix of K random initial guesses for near-nullspace of $\mathbf{A}$: $\mathbf{S^{(0)}} \in \mathbb{R}^{n\times K}$, neural model $f_\theta : \mathbb{R}^{n\times K} \to \mathbb{R}^{n\times r}$ that learns left singular vectors of a matrix and outputs the top r singular vectors, initial approximation $\mathbf{u^{(0)}} \in \mathbb{R}^n$, right-hand side $\mathbf{b} \in \mathbb{R}^n$, tolerance δ, Jacobi smoother with parameter ω.
Perform s_1 smoothing sweeps on $\mathbf{S^{(0)}}$ to obtain $\mathbf{S^{(s_1)}}$;
Compute $\hat{\mathbf{U}}_r = \mathbf{f}_\theta\left(\mathbf{S^{(s_1)}}\right)$;
Obtain orthonormal $\tilde{\mathbf{U}}_r$ via a QR-decomposition of $\hat{\mathbf{U}}_r$;
repeat
  Perform ν_1 steps of pre-smoothing on current approximation $\mathbf{u}^{(k)}$ to obtain $\tilde{\mathbf{u}}^{(k)}$;
  Compute residual $\mathbf{r}^{(k)} = \mathbf{b} - \mathbf{A}\tilde{\mathbf{u}}^{(k)}$;
  $\mathbf{A_c} = \tilde{\mathbf{U}}_r^T \mathbf{A} \tilde{\mathbf{U}}_r$;
  Project error equations to coarser level and solve the coarse-level system: $\mathbf{A_c}\mathbf{e_c} = \tilde{\mathbf{U}}_r^T \mathbf{r}^{(k)}$;
  Prolongate and add the coarse-level solution: $\tilde{\mathbf{u}}^{(k)} = \tilde{\mathbf{u}}^{(k)} + \tilde{\mathbf{U}}_r \mathbf{e_c}^{(k)}$;
  Perform ν_2 relaxation sweeps to obtain $\mathbf{u}^{(k+1)}$;
  $k = k + 1$;
until $||\mathbf{r}^{(k-1)}||_2/||\mathbf{b}||_2 < \delta$.

In our implementation, we use the Galerkin projection, with restriction operator $\mathbf{R} = \tilde{\mathbf{U}}^T$. This is a popular choice in SPD problems, but it is not a strict requirement.

5. Experiments

5.1. System Setup

Benchmarks: For our experiments, we test our loss function on a few different PDEs, each posed on the bounded domain $\mathbf{\Omega} = [0,1]^2$ and discretized using the FEM method into triangles. Firstly, there is the *diffusion equation*:

$$\begin{cases} -\nabla \cdot (g(\mathbf{x})\nabla u(\mathbf{x})) = f(\mathbf{x}), & \mathbf{x} \in \mathbf{\Omega}, \\ u(\mathbf{x}) = 0, & \mathbf{x} \in \partial\mathbf{\Omega}. \end{cases} \tag{5}$$

The goal, then, is to find $u \in H_0^1(\mathbf{\Omega})$ such that $\forall v \in H_0^1(\mathbf{\Omega})$: $a(u,v) := \int_{\mathbf{\Omega}} g\nabla u \cdot \nabla v d\mathbf{x} = \ell(v) := \int_\Omega f v d\mathbf{x}$. We then assemble the matrix $\mathbf{A}_{ij} = \sum_K g_K(\nabla\phi_i \cdot \nabla\phi_j)|K|$, with piecewise constant g_K per triangle K. For training, we choose random coefficient fields per triangle selected via $g_K \sim e^{\mathcal{N}(0,1.5)}$.

Then, we test on the *anisotropic equation*, given as:

$$\begin{cases} -\nabla\cdot(\mathbf{K}\nabla u(\mathbf{x})) = f(\mathbf{x}), & \mathbf{x} \in \mathbf{\Omega}, \\ u(\mathbf{x}) = 0, & \mathbf{x} \in \partial\mathbf{\Omega}, \end{cases} \tag{6}$$

where $\mathbf{K}(\mathbf{x}) \in \mathbb{R}^{2\times 2}$ is a positive definite tensor. We use P1 FEM element matrix with K constant on each triangle T $K_T = \int_T (\mathbf{K}_T\nabla\phi) \cdot \nabla\phi d\mathbf{x} = \mathbf{A}_T\left[(\nabla\phi_i)\mathbf{K}_T(\nabla\phi_j)\right]_{i,j=1}^3$. To construct the randomized training samples for the anisotropic equation, we first sample $\varphi \sim \mathcal{N}(0,\pi^2)$. Then, we construct a matrix:

$$\mathbf{R} = \begin{bmatrix} \cos(\varphi) & -\sin(\varphi) \\ \sin(\varphi) & \cos(\varphi) \end{bmatrix}$$

Finally, the tensor $\mathbf{K}$ is constructed via: $\mathbf{K} = \mathbf{R}\begin{bmatrix} 100000 & 0 \\ 0 & 1 \end{bmatrix}\mathbf{R}^T$. (Note: The larger we choose the top left element of the middle matrix to be, the larger the variance in our training data.) This sampling method was conducted in accordance with prior work, including (Huang, 2005; Castro-Díaz et al., 1997; Habashi et al., 2000; Remacle et al., 2005).

Finally, we test on the *screened Poisson equation* given as:

$$\begin{cases} -\nabla \cdot (\kappa\nabla u(\mathbf{x})) + \sigma u(\mathbf{x}) = f(\mathbf{x}), & \mathbf{x} \in \mathbf{\Omega}, \\ u(\mathbf{x}) = 0, & \mathbf{x} \in \partial\mathbf{\Omega}. \end{cases} \tag{7}$$

We find the weak form $u \in H_0^1(\mathbf{\Omega})$ such that $a(u,v) = \int_{\mathbf{\Omega}} \kappa\nabla u \cdot \nabla v d\mathbf{x} + \int_{\mathbf{\Omega}} \sigma u v d\mathbf{x} = \int_{\mathbf{\Omega}} f v d\mathbf{x}$. Again, we use the linear FEM (P1) with $u_h(\mathbf{x}) = \Sigma_a U_a \phi_a(\mathbf{x})$. To construct the global matrix $\mathbf{A}$, we use two separate matrices $\mathbf{M}_\sigma$ (mass/reaction element) and $\mathbf{K}_{ab}$ (stiffness/diffusion

element). These are constructed in the following manner:

$$(\mathbf{M}_\sigma)_{ab} = \sum_T \int_T \sigma_T \phi_a \phi_b dx = \sum_T (\mathbf{M}_T)_{ab},$$
$$\mathbf{K}_{ab} = \sum_T \int_T \kappa_T \nabla\phi_a \cdot \nabla\phi_b d\mathbf{x}$$
$$= \sum_T \kappa_T \mathbf{A}_T \left(\nabla\phi_a\right)^T \left(\nabla\phi_b\right).$$

The randomness is introduced here via $\kappa_T \sim e^{\mathcal{N}(0,1)}$ and an additional parameter $\alpha \sim U(0, 20)$ (it must remain non-negative to ensure the problem stays SPD). Then, we have $\mathbf{A} = \mathbf{K}_{ab} + \alpha \mathbf{M}_\sigma$.

Baselines and Metrics: First, for each listed PDE problem, we demonstrate how NLSS effectively captures the ordering of the left singular subspace in comparison to the traditional subspace loss Eq. (3). For these experiments, we test with $N = 9$ and $K = 32$ smoothed vectors where $n = (N + 1)^2$ is the dimension of our matrix discretization. For this experiment, we use a smaller architecture with a single hidden layer of 128 neurons and GELU activation, because the problem scale is also smaller.

For both NLSS and the traditional subspace loss, we train the models to full rank, thereby obtaining full approximations of the left singular subspaces. We then truncate the full matrix to smaller ranks and examine how much of the full captured energy Eq. (8) is retained:

$$\text{Captured Energy} = \frac{||\mathbf{Q}_\theta^T \mathbf{S}||_F^2}{||\mathbf{S}||_F^2}. \tag{8}$$

Note that since we are only collecting 32 smoothed vectors, 32 is our maximal rank, meaning the subspace loss and NLSS loss should, theoretically, have the same captured energy at $r = 32$. We aim to show that the NLSS retains more captured energy as ranks fall away.

Additionally, we test our learned prolongation scheme on the two-level cycle and use it as a preconditioner for PCG (Hestenes & Stiefel, 1952) to solve various PDEs. We use a tolerance of $\delta = 10^{-6}$, $\nu_1 = 5$, $\nu_2 = 5$, and $\omega = 0.66$ for our Jacobi smoother.

To compare our method with another neural method, we use the loss described in (Luz et al., 2020) to train a GNN that learns the prolongation matrix. To ensure fairness, we construct the GNN to have about the same number of parameters ($\sim$ 76.9M) as our MLP ($\sim$ 78.3M). We also compare with SA-AMG by using the PyAMG package (Bell et al., 2022) to construct the smooth aggregates and, in turn, the prolongation matrix (the same two-grid solver is used for all methods). The GNN is trained to the same rank and truncated in the same way as our spectral models to show that the loss described in (Luz et al., 2020) is not rank-agnostic. Thus, all neural models compare the same listed coarse-space dimension: n_c. The same smoother and stopping tolerance were used for all methods. We report the median total *per-instance* time, which includes inference time, setup time, accumulation of smoothed vectors $\mathbf{S}$ (when necessary), and the complete PCG process until the given tolerance was reached. Inference time is defined as the time required to construct the prolongation matrix $\mathbf{P}$. Setup time is the time required to construct the multigrid preconditioner given $\mathbf{P}$. For all neural-based methods, we first train to the maximum available rank (i.e., the number of smoothed vectors) and then truncate the learned prolongation matrix to lower ranks, as in the first experiment.

Dataset Preparation: We wanted to ensure that our training data emphasized unstructured connections. First, we chose a grid size of $(N + 1)^2$, with $N = 64$ in our particular case, on $\mathbf{\Omega} = [0, 1]^2$. Then, while locking the boundary, we introduce a jitter to randomly move nodes throughout the mesh (while ensuring that no degenerate meshes are produced). We then apply the Delaunay algorithm, which connects the jittered points so that no point lies inside the circumcircle of any triangle, and finally ensure that the resulting mesh consists only of triangles. Additional random elements are included in the details of each experiment (please see *Benchmarks*). We trained on 1000 samples and tested on 100 samples.

Implementation Details: One may note that our loss function is *architecturally agnostic*. Given this fact, we intentionally used a global MLP to minimize inference cost while optimizing performance, and we show that it already outperforms GNN baselines. So, for our PDE-solving experiments, we used a 4-layer network with 128 neurons in the 1st and 4th hidden layers, 256 neurons in the 2nd and 3rd hidden layers, and an output size of $r(N + 1)^2$ where r is the chosen rank and N is the number of columns/rows. We use the GELU activation function (Hendrycks & Gimpel, 2016), LayerNorms between hidden layers, and the Adam optimizer with a learning rate of 10^{-3}.

We also want to ensure that the ordering of the smoothed vectors does not influence the network, as we are trying to learn the left singular subspace, which is invariant to such ordering. Thus, we explicitly train with random column permutations to prevent overfitting.

We trained all of our models on the CPU (Apple M1) with 1.94GB Peak RSS. We trained for 1000 epochs utilizing the He initialization built into PyTorch. We use a constant learning rate of 10^{-3} for the MLP models and 10^{-4} for the GNN model. Finally, we used the Adam optimizer. For all PDE-solving experiments, we selected $N = 64$ and $K = 72$ smoothed vectors for all matrices.

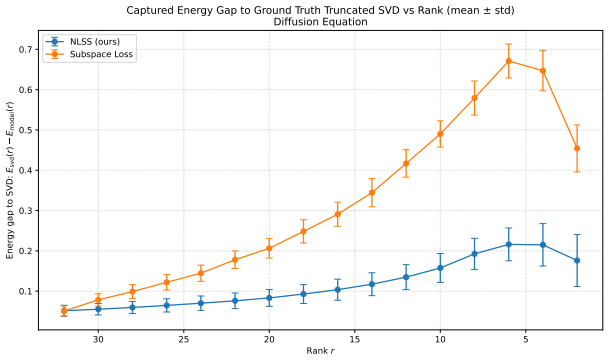

Figure 3. Difference from SVD of captured energy of NLSS vs. subspace loss for FEM diffusion equation (5).

5.2. Diffusion Equation

Fig. 3 highlights where our NLSS loss outperforms the traditional loss. We ask both models to identify the complete left subspace of the 32 smoothed vectors obtained via Jacobi smoothing across 100 test samples. However, when we begin to truncate the learned subspace to smaller and smaller ranks, we can see that the traditional subspace loss retains much less of the captured energy than our NLSS loss. If one wanted an effective left singular subspace with a smaller rank using the traditional loss, they would have to retrain the model for that specific instance. This demonstrates how our loss is *rank-agnostic*.

Table 1 shows that NLSS consistently achieves lower total time. At the $n_c = 48$ approximation, we observe that the NLSS loss solves the system more than 50% faster than the subspace loss, reinforcing the idea that our method retains the captured energy more efficiently.

5.3. Anisotropic Equation

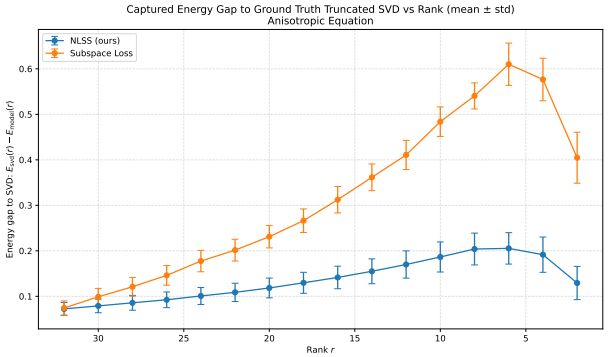

Figure 4. Difference from SVD of captured energy of NLSS vs. subspace loss for FEM anisotropic equation (6).

Again, from Fig. 4, we observe that the NLSS loss retains substantially more captured energy than the traditional subspace loss as the rank of the learned prolongation matrix decreases. The NLSS's computational advantages for solving PDEs are again highlighted for the anisotropic equation in Table 2. The conclusions remain the same as in the diffusion case.

Table 1. Results on DIFFUSION PDE. For SA-AMG, the median n_c was 493, as this was the coarse space determined by PyAMG.

(a) $n_c = 36$

	Solve Time (ms) ↓	Total (Median) (ms) ↓	Total (Q_1) (ms) ↓	Total (Q_3) (ms) ↓
Subspace	*72.11*	137.55	119.77	156.53
GNN	89.14	826.26	806.61	858.97
SVD	91.07	*122.39*	*109.11*	*137.13*
SA-AMG	100.94	157.90	146.94	174.08
NLSS (Ours)	**38.94**	**100.34**	**92.05**	**123.03**
Improvement	46%	18.02%	15.64%	10.28%

(b) $n_c = 48$

	Solve Time (ms) ↓	Total (Median) (ms) ↓	Total (Q_1) (ms) ↓	Total (Q_3) (ms) ↓
Subspace	*73.24*	139.03	121.15	166.87
GNN	94.02	835.46	816.77	865.94
SVD	101.12	*131.20*	*120.93*	*149.92*
SA-AMG	100.94	157.90	146.94	174.08
NLSS (Ours)	**35.78**	**99.35**	**88.55**	**112.79**
Improvement	51.15%	24.28%	26.78%	24.77%

(c) $n_c = 64$

	Solve Time (ms) ↓	Total (Median) (ms) ↓	Total (Q_1) (ms) ↓	Total (Q_3) (ms) ↓
Subspace	*63.23*	*130.31*	*117.10*	*147.10*
GNN	115.94	855.49	835.35	878.66
SVD	110.70	143.14	126.43	161.28
SA-AMG	100.94	157.90	146.94	174.08
NLSS (Ours)	**42.54**	**106.12**	**94.41**	**127.50**
Improvement	32.72%	18.56%	19.38%	13.32%

5.4. Screened Poisson Equation

Finally, in light of the results from Fig. 5 and Table 3, we can see that the results for both retaining captured energy and solving PDEs remain consistent across different families of highly unstructured PDEs. For additional results, please see the appendix.

Limitations: Although our approach is architecturally agnostic, there is no guarantee that other architectures will benefit to the same extent as the one we presented. GNNs, for example, may generalize to different discretization sizes, but lack the computational efficiency of a simple MLP.

Table 2. Results on ANISOTROPIC PDE. For SA-AMG, the median n_c was 493, as this was the coarse space determined by PyAMG.

(a) $n_c = 36$

	Solve Time (ms) ↓	Total (Median) (ms) ↓	Total (Q_1) (ms) ↓	Total (Q_3) (ms) ↓
Subspace	*51.49*	117.85	98.77	131.38
GNN	59.32	802.03	774.50	824.66
SVD	61.67	*92.47*	*81.10*	*106.60*
SA-AMG	90.26	151.09	138.76	180.21
NLSS (Ours)	**26.02**	**86.17**	**77.19**	**100.94**
Improvement	49.47%	6.81%	4.82%	5.31%

(b) $n_c = 48$

	Solve Time (ms) ↓	Total (Median) (ms) ↓	Total (Q_1) (ms) ↓	Total (Q_3) (ms) ↓
Subspace	*52.52*	115.80	99.34	143.96
GNN	70.15	818.54	790.12	844.76
SVD	72.27	*106.23*	*91.08*	*123.64*
SA-AMG	90.26	151.09	138.76	180.21
NLSS (Ours)	**24.41**	**85.87**	**76.07**	**103.95**
Improvement	53.52%	19.17%	16.48%	15.93%

(c) $n_c = 64$

	Solve Time (ms) ↓	Total (Median) (ms) ↓	Total (Q_1) (ms) ↓	Total (Q_3) (ms) ↓
Subspace	*48.75*	*109.36*	*96.87*	*129.73*
GNN	81.59	820.87	794.22	857.51
SVD	86.74	118.00	106.35	128.73
SA-AMG	90.26	151.09	138.76	180.21
NLSS (Ours)	**29.18**	**89.72**	**81.32**	**104.25**
Improvement	40.14%	17.96%	16.05%	19.64%

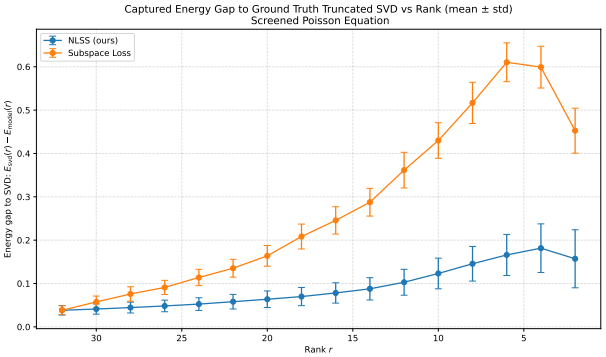

Figure 5. Difference from SVD of captured energy of NLSS vs. subspace loss for FEM screened Poisson equation (7).

6. Conclusions and Future Work

This paper presents a novel theory-grounded loss function to identify the rank-deficient left singular subspace of near-nullspace vectors for a family of discretized PDEs. We have provided a theoretical guarantee of a global minimum for this loss function. Our loss function, when combined with a neural model, enables us to avoid partitioning the matrix into aggregates while still capturing important global structure. We have shown that our model outperforms both classical and neural methods for optimizing AMG in terms of runtime while maintaining comparable convergence behavior across several challenging PDE problems. In future work, we aim to assess whether this framework can be used as a module within a larger-scale architecture to better precondition iterative schemes and, in turn, reduce the number of iterations.

Table 3. Results on SCREENED POISSON PDE. For SA-AMG, the median n_c was 493 as this was the coarse space determined by PyAMG.

(a) $n_c = 36$

	Solve Time (ms) ↓	Total (Median) (ms) ↓	Total (Q_1) (ms) ↓	Total (Q_3) (ms) ↓
Subspace	*48.87*	107.49	94.07	122.44
GNN	58.08	783.10	767.03	818.18
SVD	62.23	*93.14*	*79.42*	*112.55*
SA-AMG	71.63	127.67	117.97	155.77
NLSS (Ours)	**25.77**	**79.78**	**71.36**	**94.03**
Improvement	47.27%	14.34%	10.15%	16.45%

(b) $n_c = 48$

	Solve Time (ms) ↓	Total (Median) (ms) ↓	Total (Q_1) (ms) ↓	Total (Q_3) (ms) ↓
Subspace	*49.98*	108.24	92.55	121.82
GNN	65.44	795.79	774.08	820.39
SVD	68.57	*100.66*	*84.25*	*117.44*
SA-AMG	71.63	127.67	117.97	155.77
NLSS (Ours)	**28.68**	**83.89**	**73.22**	**97.41**
Improvement	42.62%	16.66%	13.09%	17.06%

(c) $n_c = 64$

	Solve Time (ms) ↓	Total (Median) (ms) ↓	Total (Q_1) (ms) ↓	Total (Q_3) (ms) ↓
Subspace	*45.51*	*101.96*	94.76	*116.71*
GNN	80.26	809.04	785.16	850.85
SVD	77.39	109.44	*94.43*	133.04
SA-AMG	71.63	127.67	117.97	155.77
NLSS (Ours)	**30.69**	**84.97**	**78.00**	**92.90**
Improvement	32.56%	16.66%	17.4%	20.4%

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

Bao, X., Lucas, J., Sachdeva, S., and Grosse, R. B. Regularized linear autoencoders recover the principal components, eventually. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H. (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 6971–6981. Curran Associates, Inc., 2020.

Bell, N., Olson, L. N., and Schroder, J. Pyamg: Algebraic multigrid solvers in python. *Journal of Open Source Software*, 7(72):4142, 2022.

Bochev, P. B., Garasi, C. J., Hu, J. J., Robinson, A. C., and Tuminaro, R. S. An improved algebraic multigrid method for solving maxwell's equations. *SIAM J. Sci. Comput.*, 25(2):623–642, January 2003. ISSN 1064-8275. doi: 10.1137/S1064827502407706. URL https://doi.org/10.1137/S1064827502407706.

Brandt, A. Multi-level adaptive solutions to boundary-value problems. *Mathematics of Computation*, 31(138):333–390, 1977.

Brandt, A., Brannick, J., Kahl, K., and Livshits, I. Bootstrap amg. *SIAM Journal on Scientific Computing*, 33(2):612–632, 2011.

Brenner, S. C. and Scott, L. R. *The Mathematical Theory of Finite Element Methods*, volume 15 of *Texts in Applied Mathematics*. Springer, New York, third edition, 2008. ISBN 978-0387759333.

Briggs, W. L., Henson, V. E., and McCormick, S. F. *A Multigrid Tutorial, Second Edition*. Society for Industrial and Applied Mathematics, second edition, 2000.

Castro-Díaz, M. J., Hecht, F., Mohammadi, B., and Pironneau, O. Anisotropic unstructured mesh adaption for flow simulations. *International Journal for Numerical Methods in Fluids*, 25(4):475–491, 1997.

Chow, E. An aggregation multilevel method for smooth error vectors. *SIAM Journal on Scientific Computing*, 27 (5):1727–1741, 2006.

Deng, Z., Shi, J., and Zhu, J. NeuralEF: Deconstructing kernels by deep neural networks. In Chaudhuri, K., Jegelka, S., Song, L., Szepesvari, C., Niu, G., and Sabato, S. (eds.), *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 4976–4992. PMLR, 17–23 Jul 2022.

Eckart, C. and Young, G. The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218, 1936.

Elman, H. C., Howle, V. E., Shadid, J. N., and Tuminaro, R. S. A parallel block multi-level preconditioner for the 3d incompressible navier–stokes equations. *Journal of Computational Physics*, 187(2):504–523, 2003. ISSN 0021-9991.

Falgout, R. D. An introduction to algebraic multigrid. *Computing in Science & Engineering*, 8(6):24–33, 2006.

Fan, K. On a theorem of weyl concerning eigenvalues of linear transformations i. *Proceedings of the National Academy of Sciences*, 35(11):652–655, 1949.

Frank, J. and Vuik, C. On the construction of deflation-based preconditioners. *SIAM Journal on Scientific Computing*, 23(2):442–462, 2001.

Gemp, I., McWilliams, B., Vernade, C., and Graepel, T. Eigengame: {PCA} as a nash equilibrium. In *International Conference on Learning Representations*, 2021.

Golub, G. H. and Van Loan, C. F. *Matrix Computations*. Johns Hopkins University Press, Baltimore, MD, USA, 4th edition, 2013. ISBN 9781421407944.

Greenfeld, D., Galun, M., Basri, R., Yavneh, I., and Kimmel, R. Learning to optimize multigrid PDE solvers. In Chaudhuri, K. and Salakhutdinov, R. (eds.), *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 2415–2423. PMLR, 09–15 Jun 2019.

Griebel, M., Oeltz, D., and Schweitzer, M. A. An algebraic multigrid method for linear elasticity. *SIAM Journal on Scientific Computing*, 25(2):385–407, 2003.

Habashi, W. G., Dompierre, J., Bourgault, Y., Ait-Ali-Yahia, D., Fortin, M., and Vallet, M.-G. Anisotropic mesh adaptation: towards user-independent, mesh-independent and solver-independent cfd. part i: general principles. *International Journal for Numerical Methods in Fluids*, 32(6): 725–744, 2000.

Halko, N., Martinsson, P.-G., and Tropp, J. A. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011.

Hendrycks, D. and Gimpel, K. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.

Hestenes, M. R. and Stiefel, E. Methods of conjugate gradients for solving linear systems. *Journal of Research of the National Bureau of Standards*, 49(6):409–436, 1952.

Hiptmair, R. and Xu, J. Nodal auxiliary space preconditioning in h(curl) and h(div) spaces. *SIAM Journal on Numerical Analysis*, 45(6):2483–2509, 2007.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. Lora: Low-rank adaptation of large language models, 2021.

Huang, W. Metric tensors for anisotropic mesh generation. *Journal of Computational Physics*, 204(2):633–665, 2005. ISSN 0021-9991.

Knoll, D. and Mousseau, V. On newton–krylov multigrid methods for the incompressible navier–stokes equations. *Journal of Computational Physics*, 163(1):262–267, 2000. ISSN 0021-9991.

LeVeque, R. J. *Numerical Methods for Conservation Laws*. Birkhäuser Basel, Basel, 2nd edition, 1992. ISBN 978-3-7643-2723-1.

Levinson, J., Esteves, C., Chen, K., Snavely, N., Kanazawa, A., Rostamizadeh, A., and Makadia, A. An analysis of svd for deep rotation estimation. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H. (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 22554–22565. Curran Associates, Inc., 2020.

Li, T., Shi, Z., Zhao, J., and Lin, M. Amortized eigendecomposition for neural networks. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

Luz, I., Galun, M., Maron, H., Basri, R., and Yavneh, I. Learning algebraic multigrid using graph neural networks. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 6489–6499. PMLR, 2020.

Miyato, T., Kataoka, T., Koyama, M., and Yoshida, Y. Spectral normalization for generative adversarial networks. In *International Conference on Learning Representations*, 2018.

Oftadeh, R., Shen, J., Wang, Z., and Shell, D. Eliminating the invariance on the loss landscape of linear autoencoders. In III, H. D. and Singh, A. (eds.), *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 7405–7413. PMLR, 13–18 Jul 2020.

Pernice, M. and Tocci, M. D. A multigrid-preconditioned newton–krylov method for the incompressible navier–stokes equations. *SIAM Journal on Scientific Computing*, 23(2):398–418, 2001.

Pfau, D., Petersen, S., Agarwal, A., Barrett, D. G. T., and Stachenfeld, K. L. Spectral inference networks: Unifying deep and spectral learning. In *International Conference on Learning Representations*, 2019.

Reitzinger, S. and Schöberl, J. An algebraic multigrid method for finite element discretizations with edge elements. *Numerical Linear Algebra with Applications*, 9 (3):223 – 238, April 2002. ISSN 1099-1506.

Remacle, J.-F., Li, X., Shephard, M. S., and Flaherty, J. E. Anisotropic adaptive simulation of transient flows using discontinuous galerkin methods. *International Journal for Numerical Methods in Engineering*, 62(7):899–923, 2005.

Ruge, J. W. and Stüben, K. Algebraic multigrid (amg). In McCormick, S. F. (ed.), *Multigrid Methods*, volume 3 of *Frontiers in Applied Mathematics*, pp. 73–130. SIAM, Philadelphia, PA, 1987.

Ryu, J. J., Xu, X., Erol, H. S. M., Bu, Y., Zheng, L., and Wornell, G. W. Operator svd with neural networks via nested low-rank approximation. In Salakhutdinov, R., Kolter, Z., Heller, K., Weller, A., Oliver, N., Scarlett, J., and Berkenkamp, F. (eds.), *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 42870–42905. PMLR, 21–27 Jul 2024.

Shaham, U., Stanton, K., Li, H., Nadler, B., Basri, R., and Kluger, Y. Spectralnet: Spectral clustering using deep neural networks. In *Proc. ICLR 2018*, 2018.

Stüben, K. A review of algebraic multigrid. *Journal of Computational and Applied Mathematics*, 128(1):281–309, 2001. ISSN 0377-0427. Numerical Analysis 2000. Vol. VII: Partial Differential Equations.

Thomas, J. W. *Numerical Partial Differential Equations: Finite Difference Methods*, volume 22 of *Texts in Applied Mathematics*. Springer-Verlag, New York, 1995. ISBN 978-0-387-97999-1.

Trottenberg, U., Oosterlee, C. W., and Schüller, A. *Multigrid*. Academic Press, San Diego, CA, USA, 2001. ISBN 978-0127010700.

Vaněk, P., Mandel, J., and Brezina, M. Algebraic multigrid by smoothed aggregation for second and fourth order elliptic problems. *Computing*, 56(3):179–196, 1996.

Vanek, P., Brezina, M., and Tezaur, R. Two-grid method for linear elasticity on unstructured meshes. *SIAM Journal on Scientific Computing*, 21(3):900–923, 1999.

Von Neumann, J. *Some matrix-inequalities and metrization of matric space*. 1937.

Wang, W., Dang, Z., Hu, Y., Fua, P., and Salzmann, M. Backpropagation-friendly eigendecomposition. In Wallach, H., Larochelle, H., Beygelzimer, A., d'Alché-Buc, F., Fox, E., and Garnett, R. (eds.), *Advances in Neural*

Information Processing Systems, volume 32. Curran Associates, Inc., 2019.

Whyte, T., Stathopoulos, A., and Romero, E. Chiral rank-k truncations for the multigrid preconditioner of Wilson fermions in lattice QCD. *PoS*, LATTICE2024:060, 2025.

Xu, J. and Zikatanov, L. Algebraic multigrid methods. *Acta Numerica*, 26:591–721, 2017.

A. Appendix

A.1. von Neumann Trace Inequality

For any $\mathbf{A}, \mathbf{B} \in \mathbb{C}^{n\times n}$ with singular values α_i for $\mathbf{A}$ and β_i for $\mathbf{B}$ sorted decreasingly, we have:

$$|\text{Tr}(\mathbf{AB})| \leq \sum_{i=1}^{n} \alpha_i \beta_i.$$

A.2. Proof of Theorem 4.1

Proof. First, we define $\mathcal{J}^{(k)}(\mathbf{S}, \tilde{\mathbf{P}}) = \sum_{\ell=1}^{k} ||\tilde{\mathbf{P}}_\ell^T \mathbf{S}||_F^2$. Then, minimizing $\mathcal{L}^{(k)}$ becomes equivalent to maximizing $\mathcal{J}^{(k)}$. Next, using the properties of the Frobenius norm, we have:

$$||\tilde{\mathbf{P}}_\ell^T \mathbf{S}||_F^2 = \text{Tr}(\tilde{\mathbf{P}}_\ell^T \mathbf{S}\mathbf{S}^T \tilde{\mathbf{P}}_\ell) = \sum_{j=1}^{\ell} \tilde{\mathbf{p}}_j^T \left(\mathbf{S}\mathbf{S}^T\right) \mathbf{p}_j,$$

$$\Rightarrow \mathcal{J}^{(k)} = \sum_{\ell=1}^{k} \left(\sum_{j=1}^{\ell} \tilde{\mathbf{p}}_j^T (\mathbf{S}\mathbf{S}^T) \tilde{\mathbf{p}}_j \right).$$

Now, observe that $\tilde{\mathbf{p}}_1$ appears k times, $\tilde{\mathbf{p}}_2$ appears $k-1$ times, etc. So, if we let $w_j \equiv k - j + 1$, we can rewrite our loss as:

$$\mathcal{J}^{(k)} = \sum_{j=1}^{k} w_j \tilde{\mathbf{p}}_j^T (\mathbf{S}\mathbf{S}^T) \tilde{\mathbf{p}}_j.$$

Now, let $\mathbf{A} \equiv \mathbf{S}\mathbf{S}^T$, then, our objective becomes:

$$\mathcal{J}^{(k)} = \sum_{j=1}^{k} w_j \tilde{\mathbf{p}}_j^T \mathbf{A} \tilde{\mathbf{p}}_j.$$

Now, we let $\mathbf{W} := \text{diag}(w_1, \cdots, w_k)$. Then, we can again rewrite $\mathcal{J}^{(k)}$:

$$\mathcal{J}^{(k)} = \text{Tr}(\mathbf{A}\tilde{\mathbf{P}}\mathbf{W}\tilde{\mathbf{P}}^T).$$

We can now extend the matrix $\tilde{\mathbf{P}}$ as $\mathbf{Q} = [\tilde{\mathbf{P}}, \tilde{\mathbf{P}}_\perp]$ where $\mathbf{Q}$ is now an orthogonal matrix. Finally, we define one last element, $\mathbf{B}$:

$$\mathbf{B} := \mathbf{Q} \begin{bmatrix} \mathbf{W} & 0 \\ 0 & 0 \end{bmatrix} \mathbf{Q}^T = \tilde{\mathbf{P}}\mathbf{W}\tilde{\mathbf{P}}^T,$$

$$\Rightarrow \mathcal{J}^{(k)} = \text{Tr}(\mathbf{AB}).$$

Now, by the von Neumann trace inequality for positive semidefinite matrices (Von Neumann, 1937), since the w_i's are the eigenvalues of $\mathbf{B}$, we have:

$$\mathcal{J}^{(k)} \leq \sum_{i=1}^{k} w_i \lambda_i(\mathbf{A}).$$

Finally, if $\mathbf{B}$ shares eigenvectors with $\mathbf{A}$ (i.e., when $\mathbf{Q} = \mathbf{U}$, which is the eigenbasis of $\mathbf{A}$), then this bound becomes tight. In our case, the first k columns of $\mathbf{Q}$ are exactly the top eigenvectors of $\mathbf{A}$, which correspond to the top k left singular vectors of $\mathbf{S}$. Thus, if we choose $\tilde{\mathbf{P}} = \mathbf{U_k}$, we attain the upper bound and, in turn, maximize $\mathcal{J}^{(k)}$. Moreover, if $\mathbf{P}^*$ is any global minimizer of $\mathcal{J}^{(k)}$, then we still have $\mathcal{J}^{(k)} = \sum_{i=1}^{k} w_i \lambda_i(\mathbf{A})$, i.e., span($\mathbf{P}^*$) lies in the top-$k$ eigenspace of $\mathbf{S}\mathbf{S}^T$.

□

A.3. Proof of Corollary 4.2

Proof. Let $\mathbf{S} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ and, like in the proof for Theorem 4.1, we can define $\mathbf{A} \equiv \mathbf{S}\mathbf{S}^T = \mathbf{U}\mathbf{\Sigma}^2\mathbf{U}^T$. Let $\mathbf{P} \in \mathrm{St}(n, k)$ and $\mathbf{W} = \mathrm{diag}(w_1, \cdots, w_k)$ such that $w_j = k - j + 1$. Following the steps of the proof for Theorem 4.1, we end up with:

$$\mathrm{Tr}(\mathbf{A}\mathbf{B}) \leq \sum_{i=1}^{k} \lambda_i(\mathbf{A}) w_i,$$

where $\lambda_1(\mathbf{A}) \geq \lambda_2(\mathbf{A}) \geq \cdots$ are the eigenvalues of $\mathbf{A}$. This equality will hold if and only if $\mathbf{A}$ and $\mathbf{B}$ are simultaneously diagonalizable with aligned eigenvectors (up to rotations inside eigenspaces corresponding to repeated eigenvalues). Now, we assume that $\sigma_1 > \sigma_2 > \cdots > \sigma_k > \sigma_{k+1}$ where $k < m$. Then, since $\lambda_i(\mathbf{A}) = \sigma_i^2$, we also have $\lambda_1(\mathbf{A}) > \lambda_k(\mathbf{A}) > \lambda_{k+1}(\mathbf{A})$. Thus, the eigenvectors $\mathbf{u_1}, \cdots, \mathbf{u_k}$ of $\mathbf{A}$ are uniquely determined up to a sign. So, for the equality case to hold, $\mathbf{B}$ must share the eigenvectors with $\mathbf{A}$. That is, we must have the eigendecomposition:

$$\mathbf{B} = \sum_{i=1}^{k} w_i \mathbf{u_i} \mathbf{u_i}^T.$$

However, we also know that each column $\mathbf{p_j}$ of $\mathbf{P}$ is an eigenvector of $\mathbf{B}$ with eigenvalue w_j and because w_j is a simple eigenvalue of $\mathbf{B}$, its eigenvector is unique up to a sign. Therefore, $\mathbf{p_j}$ must equal $\pm\mathbf{u_j}$ for each $j = 1, \cdots, k$, i.e.:

$$\mathbf{P}^* = \mathbf{U_k}\mathbf{D},$$

for a diagonal sign matrix $\mathbf{D}$.

□

A.4. Ablation Study on Scalability

To see how well our NLSS model scales in terms of runtime as we increase N, we compare the same architecture that we used in Section 4 against PyAMG and classic SVD. For each N, we choose $N = K$ and $r = K/2$ for both the NLSS and SVD, while PyAMG automatically computes its own coarse space. (Note: "End-to-End" time includes the generation of $\mathbf{A}$ given that we are working with different problem sizes in each experiment).

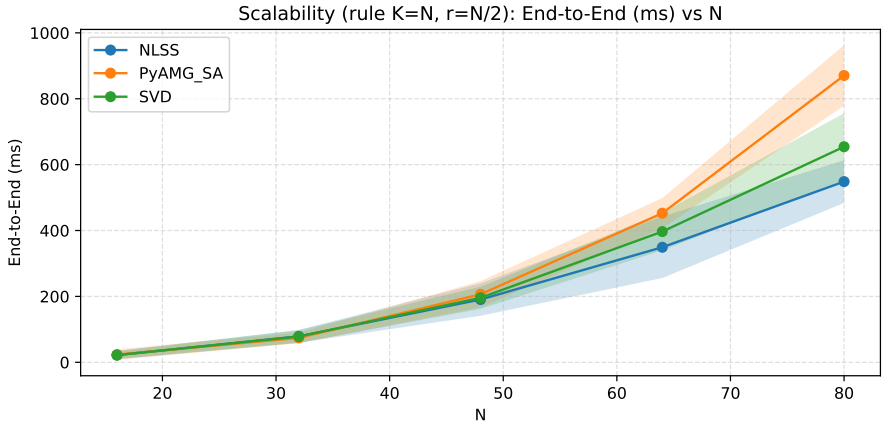

Figure 6. Comparison of runtimes for various N between NLSS, SVD, and PyAMG for the diffusion equation with triangle meshes. We choose the number of smoothed vectors to be $K = N$ and the rank $r = K/2$.

From Table 4, we can see that NLSS scales well in terms of end-to-end performance for solving PDEs as N grows. The NLSS-trained MLP outperforms SVD in inference time as N grows larger, demonstrating that it is an effective replacement for large N. Moreover, we observe that our end-to-end and solve times not only remain faster than traditional techniques, but the gap widens with increasing N.

Table 4. Ablation study of NLSS vs. PyAMG vs. SVD for solving the diffusion equation on a triangulated mesh.

N = 16

	Solve Time (ms) ↓	Setup Time (ms) ↓	Inf. Time (ms) ↓	End-to-End (Avg) (ms) ↓	End-to-End (StDev) (ms) ↓
SVD	2.77	*0.42*	*0.96*	*21.92*	**12.12**
SA-AMG ($n_c = 31$)	**1.95**	2.73	3.65	23.30	16.36
NLSS (Ours)	*2.75*	**0.40**	**0.88**	**21.81**	*12.34*

N = 32

	Solve Time (ms) ↓	Setup Time (ms) ↓	Inf. Time (ms) ↓	End-to-End (Avg) (ms) ↓	End-to-End (StDev) (ms) ↓
SVD	10.89	**0.64**	**2.03**	**78.06**	19.54
SA-AMG ($n_c = 124$)	**7.00**	2.77	4.19	78.99	**16.11**
NLSS (Ours)	*8.58*	*2.47*	*3.19*	*78.73*	*19.34*

N = 48

	Solve Time (ms) ↓	Setup Time (ms) ↓	Inf. Time (ms) ↓	End-to-End (Avg) (ms) ↓	End-to-End (StDev) (ms) ↓
SVD	*30.45*	**1.09**	**5.84**	*195.79*	**33.56**
SA-AMG ($n_c = 278$)	34.94	14.29	*7.47*	206.35	*39.98*
NLSS (Ours)	**17.19**	*3.50*	11.44	**190.54**	49.31

N = 64

	Solve Time (ms) ↓	Setup Time (ms) ↓	Inf. Time (ms) ↓	End-to-End (Avg) (ms) ↓	End-to-End (StDev) (ms) ↓
SVD	*88.82*	*2.41*	**13.58**	*396.79*	*57.09*
SA-AMG ($n_c = 491$)	113.79	46.80	*13.85*	452.58	**45.82**
NLSS (Ours)	**39.17**	**2.40**	15.58	**349.14**	93.48

N = 80

	Solve Time (ms) ↓	Setup Time (ms) ↓	Inf. Time (ms) ↓	End-to-End (Avg) (ms) ↓	End-to-End (StDev) (ms) ↓
SVD	*175.85*	**5.38**	31.41	*654.31*	101.64
SA-AMG ($n_c = 770$)	294.80	143.65	**23.38**	870.58	*92.01*
NLSS (Ours)	**73.45**	*7.19*	*26.16*	**548.47**	**64.77**

A.5. Proof of Optimality of Smooth Error Reduction

Theorem A.1. *Let $\mathbf{S} \in \mathbb{R}^{n\times m}$ be the collection of smooth error vectors. Let $\mathcal{P}_k$ be the set of all orthogonal projection matrices of rank k. Then, the projector $\mathbf{P}\mathbf{P}^T$ constructed from the global minimizer $\mathbf{P}^*$ of the NLSS loss captures the maximum possible Frobenius energy.*

Proof. Let $\mathbf{R}$ be the residual matrix after projecting the smooth vectors $\mathbf{S}$ onto the subspace spanned by $\mathbf{P}$:

$$\mathbf{R} = \mathbf{S} - \mathbf{P}\mathbf{P}^T\mathbf{S} = (\mathbf{I} - \mathbf{P}\mathbf{P}^T)\mathbf{S}.$$

We seek to minimize the total energy of the residual, defined by the squared Frobenius norm:

$$\min_{\mathbf{P}\in \mathrm{St}(n,k)} ||(\mathbf{I} - \mathbf{P}\mathbf{P}^T)\mathbf{S}||_F^2.$$

Using the property that $||\mathbf{A}||_F^2 = \mathrm{Tr}(\mathbf{A}^T\mathbf{A})$, we have:

$$||\mathbf{R}||_F^2 = \mathrm{Tr}(\mathbf{S}^T(\mathbf{I} - 2\mathbf{P}\mathbf{P}^T + \mathbf{P}\mathbf{P}^T\mathbf{P}\mathbf{P}^T)\mathbf{S}).$$

Since $\mathbf{P}^T\mathbf{P} = \mathbf{I}$, we have:

$$= \text{Tr}(\mathbf{S}^T\mathbf{S}) - \text{Tr}(\mathbf{S}^T\mathbf{P}\mathbf{P}^T\mathbf{S}).$$

Note that $\text{Tr}(\mathbf{S}^T\mathbf{S}) = ||\mathbf{S}||_F^2$ is constant, and by the cyclic property of trace, our problem becomes:

$$\arg\min_{\mathbf{P}} ||\mathbf{R}||_F^2 \Leftrightarrow \arg\max_{\mathbf{P}} \text{Tr}(\mathbf{P}^T\mathbf{S}\mathbf{S}^T\mathbf{P}).$$

Now, we know that the NLSS loss provides us with $\mathbf{P}^*$ such that $\text{span}(\mathbf{P}^*) = \text{span}(\mathbf{U_k})$ where $\mathbf{U_k}$ are the top k eigenvectors of $\mathbf{S}\mathbf{S}^T$. According to Ky Fan's maximum principle, the function $f(\mathbf{P}) = \text{Tr}(\mathbf{P}^T\mathbf{A}\mathbf{P})$ for a symmetric matrix $\mathbf{A}$, is maximized over the Stiefel manifold when the columns of $\mathbf{P}$ span the invariant subspace corresponding to the k largest eigenvalues of $\mathbf{A}$. Therefore, $\mathbf{P}^*$ maximizes the captured energy $\text{Tr}(\mathbf{P}^T\mathbf{S}\mathbf{S}^T\mathbf{P})$. Thus, $\mathbf{P}^*$ strictly minimizes the projection error $||\mathbf{S} - \mathbf{P}\mathbf{P}^T\mathbf{S}||_F^2$.

□

A.6. Galerkin Convergence

Definition A.2. The Galerkin projector is given as

$$\mathbf{\Pi_A}(\mathbf{P}) = \mathbf{P}(\mathbf{P}^T\mathbf{A}\mathbf{P})^{-1}\mathbf{P}^T\mathbf{A}.$$

Definition A.3. The *total post-correction smooth-energy error* is given as:

$$J(\mathbf{P}) := ||\mathbf{A}^{\frac{1}{2}}(\mathbf{I} - \mathbf{\Pi_A}(\mathbf{P}))\mathbf{S}||_F^2.$$

Theorem A.4. *Let $\mathbf{A}$ be SPD and let $\mathbf{S} = [\mathbf{s}_1, \cdots, \mathbf{s}_m] \in \mathbb{R}^{n\times m}$ be sampled smooth error vectors. For full-rank $\mathbf{P} \in \mathbb{R}^{n\times k}$, among rank-$k$ coarse spaces, $J(\mathbf{P})$ is minimized when the range of $\mathbf{A}^{\frac{1}{2}}\mathbf{P}$ equals the top-k left singular subspace of $\mathbf{A}^{\frac{1}{2}}\mathbf{S}$.*

Proof. Let $\mathbf{Z} := \mathbf{A}^{\frac{1}{2}}\mathbf{S}$ and $\mathbf{Q} := \mathbf{A}^{\frac{1}{2}}\mathbf{P}$. Then, we have:

$$\mathbf{\Pi_A}(\mathbf{P}) = \mathbf{A}^{-\frac{1}{2}}\mathbf{Q}(\mathbf{Q}^T\mathbf{Q})^{-1}\mathbf{Q}^T\mathbf{A}^{\frac{1}{2}}$$

$$:= \mathbf{A}^{-\frac{1}{2}}\mathbf{\Pi}(\mathbf{Q})\mathbf{A}^{\frac{1}{2}}.$$

Then, we can rewrite the objective of $J(\mathbf{P})$ as:

$$J(\mathbf{P}) = ||(\mathbf{I} - \mathbf{\Pi}(\mathbf{Q}))\mathbf{Z}||_F^2.$$

Now, we know that for orthogonal projector $\mathbf{\Pi}(\mathbf{Q})$, we have:

$$||(\mathbf{I} - \mathbf{\Pi}(\mathbf{Q}))\mathbf{Z}||_F^2 = ||\mathbf{Z}||_F^2 - ||\mathbf{\Pi}(\mathbf{Q})\mathbf{Z}||_F^2.$$

Thus, minimizing $\mathbf{J}$ is equivalent to maximizing $||\mathbf{\Pi}(\mathbf{Q})\mathbf{Z}||_F^2$. Similar to the justification in A.2, we know that the maximizing k-dimensional subspace is the span of the top-k left singular vectors of $\mathbf{Z} = \mathbf{A}^{\frac{1}{2}}\mathbf{S}$. □

It is important to note that we do not sample the left singular vectors from $\mathbf{A}^{\frac{1}{2}}\mathbf{S}$ in our work. In accordance with the work of (Chow, 2006), we approximate $\mathbf{S} \approx \mathbf{A}^{\frac{1}{2}}\mathbf{S}$. Given that we are only worried about high-energy vectors in our prolongation matrices, this is sufficient. Thus, we avoid the computation of $\mathbf{A}^{\frac{1}{2}}$ without sacrificing too much in accuracy.

A.7. Additional Results of Experiments for Varying n_c

In this section, we highlight additional experimental results for different sizes of coarse spaces, n_c, as performed in Section 4 in Tables 5, 6, and 7 for the diffusion, anisotropic, and screened Poisson equations, respectively.

Table 5. Extra results on Diffusion PDE.

$\mathbf{n_c = 24}$

	Solve Time (ms) ↓	Setup Time (ms) ↓	Total (ms) ↓	25th Percentile (ms) ↓	75th Percentile (ms) ↓
Subspace	*74.44*	**1.01**	138.29	120.27	164.13
GNN	80.43	1.13	816.95	798.40	844.94
SVD	82.67	1.18	*111.99*	**95.86**	*134.29*
SA-AMG	100.94	44.73	157.90	146.94	174.08
NLSS (Ours)	**47.38**	*1.04*	**109.52**	*97.48*	**121.56**
Improvement (%)	36.35%	-2.97%	2.21%	-1.69%	9.48%

$\mathbf{n_c = 36}$

	Solve Time (ms) ↓	Setup Time (ms) ↓	Total (ms) ↓	25th Percentile (ms) ↓	75th Percentile (ms) ↓
Subspace	*72.11*	**1.35**	137.55	119.77	156.53
GNN	89.14	1.46	826.26	806.61	858.97
SVD	91.07	1.90	*122.39*	*109.11*	*137.13*
SA-AMG	100.94	44.73	157.90	146.94	174.08
NLSS (Ours)	**38.94**	**1.35**	**100.34**	**92.05**	**123.03**
Improvement (%)	46%	0%	18.02%	15.64%	10.28%

$\mathbf{n_c = 48}$

	Solve Time (ms) ↓	Setup Time (ms) ↓	Total (ms) ↓	25th Percentile (ms) ↓	75th Percentile (ms) ↓
Subspace	*73.24*	2.22	139.03	121.15	166.87
GNN	94.02	2.00	835.46	816.77	865.94
SVD	101.12	*1.91*	*131.20*	*120.93*	*149.92*
SA-AMG	100.94	44.73	157.90	146.94	174.08
NLSS (Ours)	**35.78**	**1.80**	**99.35**	**88.55**	**112.79**
Improvement (%)	51.15%	5.76%	24.28%	26.78%	24.77%

$\mathbf{n_c = 64}$

	Solve Time (ms) ↓	Setup Time (ms) ↓	Total (ms) ↓	25th Percentile (ms) ↓	75th Percentile (ms) ↓
Subspace	*63.23*	2.60	*130.31*	*117.10*	*147.10*
GNN	115.94	**2.03**	855.49	835.35	878.66
SVD	110.70	2.25	143.14	126.43	161.28
SA-AMG	100.94	44.73	157.90	146.94	174.08
NLSS (Ours)	**42.54**	*2.13*	**106.12**	**94.41**	**127.50**
Improvement (%)	32.72%	-4.93%	18.56%	19.38%	13.32%

Table 6. Extra results on Anisotropic PDE.

$\mathbf{n_c = 24}$

	Solve Time (ms) ↓	Setup Time (ms) ↓	Total (ms) ↓	25th Percentile (ms) ↓	75th Percentile (ms) ↓
Subspace	*47.02*	*1.00*	107.23	92.91	132.15
GNN	53.29	1.02	794.58	769.42	817.14
SVD	54.20	1.15	**81.40**	**75.26**	**97.36**
SA-AMG	90.26	41.84	151.09	138.76	180.21
NLSS (Ours)	**29.11**	**0.97**	*88.58*	*80.25*	*108.90*
Improvement (%)	38.09%	3%	-8.82%	-6.63%	-11.85%

$\mathbf{n_c = 36}$

	Solve Time (ms) ↓	Setup Time (ms) ↓	Total (ms) ↓	25th Percentile (ms) ↓	75th Percentile (ms) ↓
Subspace	*51.49*	**1.27**	117.85	98.77	131.38
GNN	59.32	1.42	802.03	774.50	824.66
SVD	61.67	1.46	*92.47*	*81.10*	*106.60*
SA-AMG	90.26	41.84	151.09	138.76	180.21
NLSS (Ours)	**26.02**	*1.38*	**86.17**	**77.19**	**100.94**
Improvement (%)	49.47%	-8.66%	6.81%	4.82%	5.31%

$\mathbf{n_c = 48}$

	Solve Time (ms) ↓	Setup Time (ms) ↓	Total (ms) ↓	25th Percentile (ms) ↓	75th Percentile (ms) ↓
Subspace	*52.52*	2.18	115.80	99.34	143.96
GNN	70.15	*1.91*	818.54	790.12	844.76
SVD	72.27	2.06	*106.23*	*91.08*	*123.64*
SA-AMG	90.26	41.84	151.09	138.76	180.21
NLSS (Ours)	**24.41**	**1.80**	**85.87**	**76.07**	**103.95**
Improvement (%)	53.52%	5.76%	19.17%	16.48%	15.93%

$\mathbf{n_c = 64}$

	Solve Time (ms) ↓	Setup Time (ms) ↓	Total (ms) ↓	25th Percentile (ms) ↓	75th Percentile (ms) ↓
Subspace	*48.75*	1.99	*109.36*	*96.87*	*129.73*
GNN	81.59	*1.97*	820.87	794.22	857.51
SVD	86.74	2.42	118.00	106.35	128.73
SA-AMG	90.26	41.84	151.09	138.76	180.21
NLSS (Ours)	**29.18**	**1.95**	**89.72**	**81.32**	**104.25**
Improvement (%)	40.14%	1.02%	17.96%	16.05%	19.64%

Table 7. Extra results on Screened Poisson PDE.

$\mathbf{n_c = 24}$

	Solve Time (ms) ↓	Setup Time (ms) ↓	Total (ms) ↓	25th Percentile (ms) ↓	75th Percentile (ms) ↓
Subspace	*50.85*	**1.07**	106.56	92.77	129.67
GNN	52.69	1.26	777.92	759.20	812.66
SVD	58.52	1.24	*90.01*	**73.48**	*108.90*
SA-AMG	71.63	41.53	127.67	117.97	155.77
NLSS (Ours)	**32.67**	**1.07**	**86.41**	*76.35*	**101.89**
Improvement (%)	35.75%	0%	4%	-3.91%	6.44%

$\mathbf{n_c = 36}$

	Solve Time (ms) ↓	Setup Time (ms) ↓	Total (ms) ↓	25th Percentile (ms) ↓	75th Percentile (ms) ↓
Subspace	*48.87*	*1.45*	107.49	94.07	122.44
GNN	58.08	1.55	783.10	767.03	818.18
SVD	62.23	1.53	*93.14*	*79.42*	*112.55*
SA-AMG	71.63	41.53	127.67	117.97	155.77
NLSS (Ours)	**25.77**	**1.42**	**79.78**	**71.36**	**94.03**
Improvement (%)	47.27%	2.07%	14.34%	10.15%	16.45%

$\mathbf{n_c = 48}$

	Solve Time (ms) ↓	Setup Time (ms) ↓	Total (ms) ↓	25th Percentile (ms) ↓	75th Percentile (ms) ↓
Subspace	*49.98*	**1.84**	108.24	92.55	121.82
GNN	65.44	1.85	795.79	774.08	820.39
SVD	68.57	2.08	*100.66*	*84.25*	*117.44*
SA-AMG	71.63	41.53	127.67	117.97	155.77
NLSS (Ours)	**28.68**	*1.93*	**83.89**	**73.22**	**97.41**
Improvement (%)	42.62%	-4.89%	16.66%	13.09%	17.06%

$\mathbf{n_c = 64}$

	Solve Time (ms) ↓	Setup Time (ms) ↓	Total (ms) ↓	25th Percentile (ms) ↓	75th Percentile (ms) ↓
Subspace	*45.51*	2.02	*101.96*	94.76	*116.71*
GNN	80.26	2.29	809.04	785.16	850.85
SVD	77.39	*1.98*	109.44	*94.43*	133.04
SA-AMG	71.63	41.53	127.67	117.97	155.77
NLSS (Ours)	**30.69**	**1.95**	**84.97**	**78.00**	**92.90**
Improvement (%)	32.56%	1.52%	16.66%	17.4%	20.4%